

■■ SafeSchool — Module ELK

Toutes les étapes de l'implémentation

Dans l'ordre exact — avec le code — comme si tu l'avais fait toi-même

■ Ce document explique chaque étape que j'ai suivie pour implémenter la stack ELK dans SafeSchool, dans l'ordre chronologique, avec le code exact de chaque fichier.

Étape	Action	Fichier(s)
1	Réfléchi à l'architecture — comment NestJS va envoyer les logs vers ELK	—
2	Crée la structure de dossiers pour ELK	elk/
3	Configuré le pipeline Logstash	elk/logstash/pipeline/logstash.conf
4	Configuré Logstash (paramètres généraux)	elk/logstash/config/logstash.yml
5	Crée la politique de rétention des logs (30 jours)	elk/setup/ilm-policy.json
6	Crée le script d'initialisation Elasticsearch	elk/setup/setup.sh
7	Ajouté les 4 services ELK dans Docker	docker-compose.yml
8	Crée le transport TCP Winston → Logstash	backend/src/logger/logstash.transport.ts
9	Crée le service de logging central	backend/src/logger/logger.service.ts
10	Crée le middleware de logging HTTP	backend/src/logger/http-logger.middleware.ts
11	Crée le module NestJS du logger	backend/src/logger/logger.module.ts
12	Intégré le logger dans l'application NestJS	main.ts + app.module.ts

13	Ajouté les logs dans auth.service.ts	backend/src/auth/auth.service.ts
14	Ajouté les logs dans reports.service.ts	backend/src/reports/reports.service.ts
15	Remplacé les console.log dans scoring.service.ts	backend/src/reports/scoring.service.ts
16	Ajouté les dépendances winston	backend/package.json
17	Ajouté les variables d'environnement ELK	.env.example + .env
18	Corrigé les Dockerfiles pour éviter les problèmes de cache	backend/Dockerfile + frontend/Dockerfile

Étape 1 — Réflexion sur l'architecture

Avant d'écrire la moindre ligne de code, j'ai réfléchi à comment les logs allaient circuler dans le système. La question principale était : comment NestJS va-t-il envoyer ses logs vers Logstash ?

J'ai choisi une connexion TCP directe entre NestJS et Logstash. NestJS envoie chaque log en JSON sur une ligne (format `json_lines`) via TCP sur le port 5044. Logstash écoute sur ce port, enrichit les logs, et les envoie vers Elasticsearch.



J'ai aussi décidé ce que j'allais logger : toutes les requêtes HTTP (via un middleware), les événements d'authentification (connexions réussies et échouées), la création et mise à jour des signalements, et le détail du calcul de score IA.

Étape 2 — Création de la structure de dossiers

J'ai créé un dossier elk/ à la racine du projet pour regrouper toute la configuration ELK. Cette organisation permet de tout retrouver facilement et sépare clairement la config ELK du reste du projet.

■ Structure de dossiers créée

```
# Commande exécutée pour créer les dossiers
mkdir -p elk/logstash/pipeline
mkdir -p elk/logstash/config
mkdir -p elk/setup

# Structure finale obtenue :
elk/
  ■■■ logstash/
    ■ ■■■ pipeline/
      ■ ■■■ logstash.conf ← pipeline de traitement des logs
    ■ ■■■ config/
      ■ ■■■ logstash.yml ← configuration générale Logstash
    ■■■ setup/
      ■■■ ilm-policy.json ← politique de rétention 30 jours
      ■■■ setup.sh ← script d'initialisation au démarrage
```

Étape 3 — Configuration du pipeline Logstash

J'ai créé le fichier `logstash.conf` qui définit comment Logstash traite les logs. Ce fichier a 3 sections obligatoires : `input` (comment les logs arrivent), `filter` (comment on les transforme) et `output` (où on les envoie).

```
■ elk/logstash/pipeline/logstash.conf

# SECTION 1 — INPUT
# Logstash écoute les connexions TCP sur le port 5044
# Chaque message reçu est une ligne JSON (codec json_lines)
input {
  tcp {
    port => 5044
    codec => json_lines
  }
}

# SECTION 2 — FILTER
# On enrichit les logs avant de les envoyer à Elasticsearch
filter {
  # Met le level en majuscules : info -> INFO
  if [level] { mutate { uppercase => ['level'] } }

  # Parse le timestamp NestJS comme vrai timestamp Elasticsearch
  if [timestamp] {
    date {
      match => ['timestamp', 'ISO8601']
      target => '@timestamp'
      remove_field => ['timestamp']
    }
  }

  # Ajoute des tags pour filtrer facilement dans Kibana
  if [type] == 'http_request' { mutate { add_tag => ['http'] } }
  if [type] == 'auth_event' { mutate { add_tag => ['auth'] } }
  if [type] == 'report_event' { mutate { add_tag => ['report'] } }
  if [level] == 'ERROR' { mutate { add_tag => ['error'] } }
}

# SECTION 3 — OUTPUT
# Envoie vers Elasticsearch — un index par jour
# safeschool-logs-2026.04.15, safeschool-logs-2026.04.16...
output {
  elasticsearch {
```

```
hosts => ['http://elasticsearch:9200']
index => 'safeschool-logs-%{+YYYY.MM.dd}'
}
stdout { codec => rubydebug } # Aussi dans la console Docker
}
```

■ Un index par jour (safeschool-logs-2026.04.15) facilite la gestion : pour supprimer les logs de plus de 30 jours, Elasticsearch supprime simplement l'index du jour J-30 — c'est instantané.

Étape 4 — Configuration générale de Logstash

J'ai créé le fichier `logstash.yml` avec la configuration minimale nécessaire : le port de l'API de monitoring et la désactivation du monitoring xpack (qui nécessiterait une licence payante).

```
■ elk/logstash/config/logstash.yml
# elk/logstash/config/logstash.yml

# Port de l'API HTTP de monitoring de Logstash
# Permet de vérifier l'état de Logstash via http://localhost:9600
http.host: '0.0.0.0'
http.port: 9600

# Désactive le monitoring X-Pack
# (fonctionnalité payante d'Elasticsearch, pas nécessaire en dev)
xpack.monitoring.enabled: false
```

Étape 5 — Politique de rétention des logs (30 jours)

J'ai créé la politique ILM (Index Lifecycle Management) qui définit le cycle de vie des logs. Les logs passent par 3 phases : hot (actifs, 7 jours), warm (archivés, 23 jours), puis sont supprimés automatiquement à 30 jours. Cela répond à l'exigence du sujet 'Implement log retention and archiving policies'.

■ elk/setup/ilm-policy.json

```
// elk/setup/ilm-policy.json
{
  "policy": {
    "phases": {

      // PHASE HOT – logs actifs des 7 derniers jours
      // Elasticsearch fait un rollover (nouvel index) si > 7 jours ou > 10GB
      "hot": {
        "min_age": "0ms",
        "actions": {
          "rollover": { "max_age": "7d", "max_size": "10gb" }
        }
      },

      // PHASE WARM – logs archivés de 7 à 30 jours
      // Les données sont compressées (forcemerge) pour économiser l'espace
      "warm": {
        "min_age": "7d",
        "actions": {
          "shrink": { "number_of_shards": 1 },
          "forcemerge": { "max_num_segments": 1 }
        }
      },

      // PHASE DELETE – suppression automatique à 30 jours
      "delete": {
        "min_age": "30d",
        "actions": { "delete": {} }
      }
    }
  }
}
```

Étape 6 — Script d'initialisation Elasticsearch

J'ai créé un script shell qui s'exécute une seule fois au démarrage pour configurer Elasticsearch : créer la politique ILM et le template d'index. Ce script tourne dans un conteneur Docker dédié (elk-setup) qui attend qu'Elasticsearch soit prêt avant de s'exécuter.

■ elk/setup/setup.sh

```
#!/bin/sh
# elk/setup/setup.sh

# Attend qu'Elasticsearch soit prêt (healthcheck)
echo 'Attente d Elasticsearch...'
until curl -s http://elasticsearch:9200/_cluster/health > /dev/null; do
sleep 2
done
echo 'Elasticsearch est pret !'

# 1. Crée la politique de rétention (30 jours)
curl -X PUT 'http://elasticsearch:9200/_ilm/policy/safeschool-logs-policy' \
-H 'Content-Type: application/json' \
-d @/setup/ilm-policy.json

# 2. Crée le template d'index
# Tous les index safeschool-logs-* héritent automatiquement de la politique ILM
curl -X PUT 'http://elasticsearch:9200/_index_template/safeschool-logs-template' \
-H 'Content-Type: application/json' \
-d '{
  "index_patterns": ["safeschool-logs-*"],
  "template": {
    "settings": {
      "index.lifecycle.name": "safeschool-logs-policy",
      "number_of_shards": 1,
      "number_of_replicas": 0
    }
  }
}'

echo 'Configuration ELK terminee ! Kibana : http://localhost:5601'
```

Étape 7 — Ajout des 4 services ELK dans Docker

J'ai modifié docker-compose.yml pour ajouter les 4 nouveaux services. J'ai aussi ajouté depends_on dans le service backend pour qu'il attende que Logstash soit démarré avant de lancer NestJS.

■ docker-compose.yml – nouveaux services

```
# Ajout dans docker-compose.yml

# SERVICE 1 – Elasticsearch (base de données des logs)
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.11.0
  container_name: safeschool_elasticsearch
  environment:
    - discovery.type=single-node # Pas de cluster, mode dev
    - xpack.security.enabled=false # Sécurité désactivée en dev
    - ES_JAVA_OPTS=-Xms512m -Xmx512m # Limite RAM
  ports:
    - '9200:9200'
  volumes:
    - esdata:/usr/share/elasticsearch/data # Persistance
  healthcheck:
    test: ['CMD-SHELL', 'curl -f http://localhost:9200/_cluster/health']
    interval: 15s
    retries: 10

# SERVICE 2 – Logstash (collecteur de logs)
logstash:
  image: docker.elastic.co/logstash/logstash:8.11.0
  container_name: safeschool_logstash
  ports:
    - '5044:5044' # NestJS envoie les logs ici
  volumes:
    - ./elk/logstash/pipeline:/usr/share/logstash/pipeline
    - ./elk/logstash/config/logstash.yml:/usr/share/logstash/config/logstash.yml
  depends_on:
    elasticsearch:
      condition: service_healthy

# SERVICE 3 – Kibana (visualisation)
kibana:
  image: docker.elastic.co/kibana/kibana:8.11.0
  container_name: safeschool_kibana
  ports:
```

```
- '5601:5601'
environment:
- ELASTICSEARCH_HOSTS=http://elasticsearch:9200
depends_on:
elasticsearch:
condition: service_healthy

# SERVICE 4 – Setup (tourne une seule fois au démarrage)
elk-setup:
image: curlimages/curl:latest
volumes:
- ./elk/setup:/setup
depends_on:
elasticsearch:
condition: service_healthy
command: sh /setup/setup.sh

# Ajout du volume pour persister les données Elasticsearch
volumes:
pgdata: # existait déjà
esdata: # NOUVEAU
```

Étape 8 — Transport TCP Winston → Logstash

J'ai créé un transport personnalisé pour Winston. Un 'transport' c'est la destination où Winston envoie les logs. J'ai créé le fichier `logstash.transport.ts` qui ouvre une connexion TCP vers Logstash et envoie chaque log en JSON. J'ai aussi géré la reconnexion automatique et la mise en tampon des messages.

■ `backend/src/logger/logstash.transport.ts`

```
// backend/src/logger/logstash.transport.ts

// On étend TransportStream (classe de base de Winston pour les transports)
export class LogstashTransport extends TransportStream {
  private client: net.Socket | null = null;
  private connected = false;
  private buffer: string[] = []; // Tampon si Logstash pas encore dispo

  constructor(opts: { host: string; port: number }) {
    super(opts);
    this.connect(); // Connexion au démarrage
  }

  private connect(): void {
    this.client = new net.Socket();
    this.client.connect(this.port, this.host, () => {
      this.connected = true;
      // Connexion établie → vide le tampon
      this.buffer.forEach(msg => this.client!.write(msg));
      this.buffer = [];
    });
    this.client.on('error', () => {
      this.connected = false;
      setTimeout(() => this.connect(), 5000); // Réessaie dans 5 sec
    });
  }

  // Appelée par Winston pour chaque log à envoyer
  log(info: any, callback: () => void): void {
    const message = JSON.stringify({
      ...info,
      application: 'safeschool-backend',
      timestamp: new Date().toISOString(),
    }) + '\n'; // \n important : json_lines = une ligne = un log

    if (this.connected) {
      this.client!.write(message); // Envoi direct
    } else {

```

```
this.buffer.push(message); // Mise en tampon  
}  
callback();  
}  
}
```

Étape 9 — Le service de logging central

J'ai créé `logger.service.ts` qui est le cœur du système. Ce service crée un logger Winston avec 2 transports : Console (pour voir les logs dans Docker) et Logstash (pour les envoyer vers Kibana). Il expose aussi des méthodes spécialisées pour chaque type d'événement SafeSchool.

■ `backend/src/logger/logger.service.ts`

```
// backend/src/logger/logger.service.ts

@Injectable()
export class LoggerService implements NestLoggerService {
  private readonly logger: winston.Logger;

  constructor() {
    const transports: winston.transport[] = [
      // Transport 1 : Console (toujours actif)
      new winston.transports.Console({ format: ... }),
    ];

    // Transport 2 : Logstash (seulement si LOGSTASH_HOST défini dans .env)
    if (process.env.LOGSTASH_HOST) {
      transports.push(new LogstashTransport({
        host: process.env.LOGSTASH_HOST, // 'logstash' dans Docker
        port: parseInt(process.env.LOGSTASH_PORT || '5044'),
      }));
    }

    this.logger = winston.createLogger({ level: 'info', transports });
  }

  // Méthodes standard NestJS (obligatoires pour implémenter NestLoggerService)
  log(message: string, context?: string) { this.logger.info(message, { context }); }
  error(message: string, trace?: string) { this.logger.error(message, { trace }); }
  warn(message: string) { this.logger.warn(message); }

  // Méthodes spécialisées SafeSchool
  http(data: { type: 'http_request'; method: string; url: string; ... }) {
    this.logger.info(`${data.method} ${data.url} -> ${data.statusCode}`, data);
  }

  auth(data: { type: 'auth_event'; action: string; email: string; ... }) {
    const level = data.action === 'login_failure' ? 'warn' : 'info';
    this.logger.log(level, `[Auth] ${data.action}`, data);
  }

  report(data: { type: 'report_event'; action: string; ... }) {
    this.logger.info(`[Report] ${data.action}`, data);
  }
}
```

```
scoring(data: { type: 'scoring_event'; finalScore: number; grade: string; ... }) {  
  this.logger.info(`[Scoring] ${data.finalScore}/100 -> ${data.grade}`, data);  
}  
}
```

Étape 10 — Middleware de logging HTTP automatique

J'ai créé un middleware NestJS qui s'applique automatiquement sur toutes les routes. Il log chaque requête HTTP après que la réponse est envoyée. L'avantage d'écouter l'événement 'finish' c'est qu'on a accès au code de statut et au temps de réponse.

```
■ backend/src/logger/http-logger.middleware.ts

// backend/src/logger/http-logger.middleware.ts

@Injectable()
export class HttpLoggerMiddleware implements NestMiddleware {
  constructor(private readonly logger: LoggerService) {}

  use(req: Request, res: Response, next: NextFunction): void {
    const { method, originalUrl, ip } = req;
    const start = Date.now(); // Chronomètre

    // On attend que la réponse soit envoyée
    // (pour avoir le statusCode et le temps de réponse)
    res.on('finish', () => {
      const user = (req as any).user; // Injecté par Passport JWT

      this.logger.http({
        type: 'http_request',
        method,
        url: originalUrl,
        statusCode: res.statusCode,
        responseTime: `${Date.now() - start}ms`,
        userId: user?.id || 'anonymous',
        userRole: user?.role || 'anonymous',
        ip: ip || 'unknown',
      });
    });

    next(); // Continue vers le contrôleur
  }
}
```

Étape 11 — Module NestJS du logger

J'ai créé le module NestJS pour enregistrer LoggerService. J'ai utilisé le décorateur `@Global()` pour que LoggerService soit injectable dans tous les autres modules sans avoir à réimporter LoggerModule partout.

```
■ backend/src/logger/logger.module.ts

// backend/src/logger/logger.module.ts

// @Global() = disponible dans toute l'application
// sans avoir à importer LoggerModule dans chaque module
@Global()
@Module({
  providers: [LoggerService],
  exports: [LoggerService], // Exporté pour être injecté ailleurs
})
export class LoggerModule {}
```

Étape 12 — Intégration dans l'application NestJS

J'ai modifié main.ts pour utiliser notre LoggerService comme logger principal de NestJS (à la place du logger par défaut). J'ai aussi modifié app.module.ts pour importer LoggerModule et appliquer le middleware HTTP sur toutes les routes.

■ backend/src/main.ts

```
// backend/src/main.ts – MODIFICATIONS

async function bootstrap() {
  const app = await NestFactory.create(AppModule, {
    bufferLogs: true, // Buffer les logs le temps que Winston soit prêt
  });

  // Récupère notre LoggerService et l'utilise comme logger NestJS
  // Maintenant tous les logs internes NestJS passent aussi par Winston
  const logger = app.get(LoggerService);
  app.useLogger(logger);

  app.enableCors({ ... });
  await app.listen(process.env.PORT ?? 3000);

  logger.log('SafeSchool backend démarré');
  logger.log(`Logstash : ${process.env.LOGSTASH_HOST || 'désactivé'}`);
}
```

■ backend/src/app.module.ts

```
// backend/src/app.module.ts – MODIFICATIONS

// AppModule implémente NestModule pour pouvoir configurer les middlewares
@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    TypeOrmModule.forRoot({ ... }),
    LoggerModule, // AJOUTÉ – module de logging global
    AuthModule,
    UsersModule,
    ReportsModule,
    NotificationsModule,
    StudentProfilesModule,
  ],
})
export class AppModule implements NestModule {
  // AJOUTÉ – applique le middleware HTTP sur TOUTES les routes (*)
  configure(consumer: MiddlewareConsumer) {
```

```
consumer.apply(HttpLoggerMiddleware).forRoutes('*');  
}  
}
```

Étape 13 — Logs dans auth.service.ts

J'ai injecté LoggerService dans AuthService et ajouté 3 appels de logs : échec connexion (email inconnu), échec connexion (mauvais mot de passe), et connexion réussie. Ces logs sont essentiels pour détecter les tentatives d'intrusion.

■ backend/src/auth/auth.service.ts

```
// backend/src/auth/auth.service.ts – MODIFICATIONS

// 1. Injection du logger dans le constructeur
constructor(
  private userService: UserService,
  private jwtService: JwtService,
  private logger: LoggerService, // AJOUTÉ
) {}

async login(email: string, password: string) {
  const user = await this.userService.findByEmailWithProfile(email);

  if (!user) {
    // AJOUTÉ – log email inconnu
    this.logger.auth({ type: 'auth_event', action: 'login_failure',
      email, reason: 'email_not_found' });
    throw new UnauthorizedException('...');
  }

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) {
    // AJOUTÉ – log mauvais mot de passe
    this.logger.auth({ type: 'auth_event', action: 'login_failure',
      email, userId: user.id, userRole: user.role, reason: 'wrong_password' });
    throw new UnauthorizedException('...');
  }

  // AJOUTÉ – log connexion réussie
  this.logger.auth({ type: 'auth_event', action: 'login_success',
    email, userId: user.id, userRole: user.role });

  return { access_token: ..., user: ... };
}
```

Étape 14 — Logs dans reports.service.ts

J'ai ajouté 2 logs dans reports.service.ts : après la création d'un signalement (avec son grade et son score) et après chaque mise à jour de statut. Ces logs permettent de suivre l'historique complet de chaque dossier.

```
■ backend/src/reports/reports.service.ts

// backend/src/reports/reports.service.ts — MODIFICATIONS

// 1. Injection du logger
constructor(
  @InjectRepository(Rreport) private reportsRepository: ...,
  ...
  private logger: LoggerService, // AJOUTÉ
) {}

async create(...): Promise {
  // ... code existant de création du signalement ...

  // AJOUTÉ — log après création
  this.logger.report({
    type: 'report_event',
    action: 'created',
    reportId: savedReport.id,
    caseNumber, // ex: #2026-009
    grade, // ex: critique
    score: finalScore, // ex: 72
    status: ReportStatus.PENDING,
    userId: student.id,
    userRole: student.role,
  });

  return finalReport;
}

async update(id, updates): Promise {
  // ... code existant de mise à jour ...

  const saved = await this.reportsRepository.save(report);

  // AJOUTÉ — log après mise à jour
  this.logger.report({
    type: 'report_event',
    action: 'updated',
    reportId: id,
    caseNumber: report.caseNumber,
    grade: saved.grade,
```

```
status: saved.status,  
});  
return saved;  
}
```

Étape 15 — Remplacement des console.log dans scoring.service.ts

Le scoring.service.ts avait plusieurs console.log() pour déboguer le calcul de score. Je les ai remplacés par des appels au logger structuré. Maintenant le détail complet de chaque calcul (tous les critères) est visible dans Kibana.

```
■ backend/src/reports/scoring.service.ts

// backend/src/reports/scoring.service.ts — MODIFICATIONS

// AVANT (console.log dispersés, pas structurés)
console.log(`[Scoring] type:${typeScore} | base:${baseScore}`);
console.log(`[Scoring] IA:${aiScore} | urgence:${urgency}`);
console.log(`[Scoring] Score final : ${finalScore}`);
console.log(`[Scoring] Grade : ${grade}`);

// APRÈS (un seul log structuré avec TOUS les détails)
this.logger.scoring({
  type: 'scoring_event',
  typeScore, // 25 (physique)
  frequencyScore, // 20 (tous les jours)
  classScore, // 7 (5ème)
  recidiveScore, // 0 (premier signalement)
  suspectRoleScore, // 0 (élève)
  aiScore, // 20 (menace physique détectée)
  finalScore, // 72
  grade, // critique
  aiReason, // 'Menace physique détectée'
  urgency, // true
});
```

■ Maintenant dans Kibana on peut chercher : type : scoring_event AND grade : critique pour voir tous les signalements critiques avec le détail de leur calcul de score.

Étapes 16, 17, 18 — Dépendances, variables, Dockerfiles

Étape 16 — Ajout des packages Winston dans package.json

J'ai ajouté les deux packages nécessaires dans backend/package.json. En ajoutant ces packages ici, ils sont automatiquement installés lors du docker compose up --build sans aucune intervention manuelle.

```
■ backend/package.json

// backend/package.json – AJOUTS dans dependencies
"dependencies": {
  ...
  "winston": "^3.11.0", // Librairie de logging
  "winston-transport": "^4.7.0", // Classe de base pour les transports
  ...
}
```

Étape 17 — Variables d'environnement ELK

J'ai ajouté 3 nouvelles variables dans .env.example et .env. LOGSTASH_HOST indique à NestJS où envoyer les logs. Sans cette variable, le logger fonctionne mais n'envoie rien à Logstash (mode console uniquement).

```
■ .env.example + .env

# .env et .env.example – AJOUTS

# Nom du conteneur Logstash dans le réseau Docker
LOGSTASH_HOST=logstash

# Port TCP d'écoute de Logstash (doit correspondre à logstash.conf)
LOGSTASH_PORT=5044

# Niveau minimum de log : error | warn | info | debug
LOG_LEVEL=info
```

Étape 18 — Correction des Dockerfiles

J'ai rencontré un problème : quand on fait docker compose up --build, Docker utilise un volume anonyme pour node_modules qui peut être en cache avec d'anciennes versions des packages. Pour garantir que tout s'installe correctement sur n'importe quel ordi, j'ai modifié le CMD des Dockerfiles pour lancer npm install au démarrage.

```
■ backend/Dockerfile + frontend/Dockerfile

# backend/Dockerfile – MODIFICATION du CMD

# AVANT
```

```
CMD ["npm", "run", "start:dev"]
```

```
# APRÈS – npm install au démarrage pour toujours avoir les bons packages
```

```
# --prefer-offline utilise le cache si le package est déjà là (rapide)
```

```
# Si un nouveau package est dans package.json, il est téléchargé automatiquement
```

```
CMD ["sh", "-c", "npm install --prefer-offline && npm run start:dev"]
```

```
# Même modification dans frontend/Dockerfile
```

```
CMD ["sh", "-c", "npm install --prefer-offline && npm run dev -- --host"]
```

Résumé — Ce que fait le système ELK dans SafeSchool

Ce qui se passe	Fichier responsable	Log envoyé
Un utilisateur se connecte avec succès	auth.service.ts	type:auth_event action:login_success level:INFO
Un utilisateur tape un mauvais mot de passe	auth.service.ts	type:auth_event action:login_failure level:WARN
N'importe quelle requête HTTP est reçue	http-logger.middleware.ts	type:http_request method url statusCode responseTime
Un élève crée un signalement	reports.service.ts	type:report_event action:created grade score caseNumber
Un admin modifie le statut d'un dossier	reports.service.ts	type:report_event action:updated status grade
L'IA calcule le score d'un signalement	scoring.service.ts	type:scoring_event tous les critères finalScore grade urgency
NestJS démarre et charge ses modules	main.ts (logger global)	level:INFO context:InstanceLoader message:Module initialized
Une erreur se produit dans le backend	N'importe quel service	level:ERROR message trace (stack trace)

■ Pour démarrer : `docker compose up --build` — puis `http://localhost:5601` → Discover → Create data view → `safeschool-logs-*` → `@timestamp` → Save